M21 Debug Méthodique

Méthodologie Intermediate

Approche structurée pour diagnostiquer et résoudre les erreurs

Le problème de l'itération aveugle

La tentation est forte de décrire un bug vaguement et de laisser Claude essayer des corrections en série. Ce pattern est coûteux en tokens et en temps, et produit souvent des « corrections » qui masquent le problème sans le résoudre. Une approche structurée en trois phases change radicalement l'efficacité du debug.

Phase 1 : Reproduire

Avant d'analyser ou de corriger quoi que ce soit, obtenir une reproduction fiable. Un bug qu'on ne peut pas reproduire de façon déterministe est un bug qu'on ne peut pas corriger de façon fiable.

Fournir à Claude le minimum nécessaire pour reproduire :

- Message d'erreur exact (copier-coller, pas un résumé)
- Stack trace complète si disponible
- Commande ou séquence d'actions qui déclenchent l'erreur
- Environnement : OS, version de Node/Python/etc., variables d'env pertinentes

Phase 2 : Isoler

Créer un cas de test minimal qui reproduit le bug. L'isolation sert à deux choses : elle confirme qu'on a bien compris la cause, et elle fournit un test de régression immédiat une fois le bug corrigé.

```
# Exemple : isoler un bug de parsing
node -e "
const parse = require('./src/parser');
const input = 'cas problématique';
console.log(parse(input));
"
```

Demander à Claude d'expliquer son hypothèse avant chaque changement. Si Claude ne peut pas articuler pourquoi il modifie un fichier, c'est un signal d'itération aveugle.

Phase 3 : Corriger et vérifier

Une fois la cause isolée, appliquer la correction minimale. Éviter de profiter du contexte de debug pour refactorer du code adjacent : le scope creep complique la validation.

Vérification en deux temps : le cas de test minimal passe, puis la suite de tests complète passe. Si un test existant casse après la correction, c'est que la correction a des effets de bord non anticipés.

Prompt de debug structuré

```
Bug : [description en une phrase]
Erreur exacte : [message + stack trace]
Reproduction : [commande / étapes]
Déjà essayé : [tentatives précédentes]

Avant de modifier quoi que ce soit,
énumère tes hypothèses par ordre de probabilité.
```

Ce prompt force Claude à exposer son raisonnement, ce qui permet de valider ou corriger la direction avant qu'il n'écrive la moindre ligne de code.

Quand utiliser des agents de debug

Un agent unique convient pour les bugs localisés sur 1 à 3 fichiers. Pour les problèmes qui traversent plusieurs composants interconnectés (un bug de réseau qui affecte l'auth qui affecte la base de données), déléguer l'exploration à un sous-agent via l'Agent tool (anciennement Task) préserve le contexte principal.

```
Main agent → Agent: explore auth flow from request to DB
               └─ Sub-agent explore et retourne un résumé
Main agent reçoit le résumé (pas le contexte brut)
```

Un sous-agent retourne un résumé plus court, mais ses erreurs peuvent influencer l'agent principal. Vérifier les constats qui ont des conséquences.

Signaux d'alarme pendant le debug

Signal	Interprétation
Claude modifie un fichier sans explication	Demander « pourquoi ce fichier ? »
Correction qui contourne l'erreur	Chercher la cause racine, pas le symptôme
Tests qui passent mais comportement inchangé	Test inadapté au bug réel
Même erreur après correction	Reproduction insuffisamment isolée

Réinitialiser le contexte

Un contexte long ou non pertinent peut rendre les contraintes importantes plus difficiles à appliquer ; aucun nombre fixe de tours ne le prédit. Si le débogage s'éternise, `/compact` résume l'historique et `/clear` repart avec les seuls éléments essentiels.